
COMPASS-SIG

设计规范与支撑文档

版本： 1.2

日期： 2026 年 6 月

作者：王更，夏雯雯，汪哲东，季炜丹，何文字，张宁岳，段非，耿欣阳，李继红

单位：上海交通大学密码与计算机安全实验室；上海智巡密码科技有限公司

状态：规范草案

目录

1	引言	1
2	算法描述	2
2.1	符号与记号	2
2.2	签名方案概览	4
2.2.1	算法参数	4
2.2.2	NTT 域表示	5
2.2.3	哈希	5
2.2.4	拆分与压缩	6
2.3	核心算法	7
2.3.1	密钥生成	7
2.3.2	签名	8
2.3.3	验证	10
3	设计原理	11
3.1	安全基础	11
3.2	安全模型	12
3.3	参数集	13
3.4	失败分析与重复次数	14
3.5	差异性声明	15
4	安全声明与分析	15
4.1	理论安全性	15
4.2	具体安全性	17
4.2.1	密钥恢复的具体安全性	17
4.2.2	签名伪造的具体安全性	19
4.2.3	强签名伪造的具体安全性	22
5	性能	23
5.1	性能分析	23
5.1.1	实现考量与权衡	23
5.1.2	跨安全级别的性能扩展	24
5.2	参考实现	24
5.2.1	x86 架构上的性能结果	24
5.2.2	AVX2 优化实现	25

1. 引言

本规范定义了 COMPASS-SIG，即 Compact Post-quantum Security Suite（紧凑后量子安全套件）的一部分。它是一种基于模格困难问题的后量子数字签名方案（SIG）。该算法旨在对抗大规模量子计算机带来的安全威胁，支持四种安全类别，分别对应 128 比特（80 比特量子安全）、256 比特（128 比特量子安全）、384 比特（192 比特量子安全）和 512 比特（256 比特量子安全）。

本算法的设计准则列举如下：

- **高效且易于实现的 Fiat-Shamir with Abort 范式。**在所有后量子签名范式（Fiat-Shamir with Abort、Hash-and-Sign、MPC/VOLE-in-the-Head 等）中，Fiat-Shamir with Abort（FSwA）范式实现简单，无需复杂的子模块如陷门生成或陷门采样。特别是，当配备均匀误差 LWE 时，FSwA 具有确定性拒绝采样，可在常数时间内高效完成，从而免受侧信道攻击。
- **避免参数中不必要的冗余。**最著名的具有确定性拒绝的 FSwA 格签名是 NIST 标准 ML-DSA（Dilithium）。然而，我们指出 ML-DSA 在其设计中存在不必要的冗余，主要源于 MSIS 估计的保守性。在 ML-DSA 中，攻破 MSIS 仅导致签名伪造，而攻破 MLWE 意味着完全恢复秘密密钥，这自然蕴含签名伪造。因此 MSIS 估计中的保守性对 ML-DSA 的安全性没有影响，反而导致了参数选择中的冗余。

替代 ML-DSA 中的保守估计，我们提出了一种新颖的设计，提供了针对签名伪造的紧致安全性估计，即针对 MSIS/MISIS 的困难性，并使用我们全新设计的安全评估工具进行安全估计。此外，我们提出了一种针对强不可伪造性的新颖安全评估，并证明强伪造的附加条件实际上比普通伪造更难实现，这与 ML-DSA 的估计结果所隐含的不同。我们注意到，我们估计中使用的某些技术在 ML-DSA 提出时尚不存在。

- **优化通信代价。**ML-DSA 的设计理念是最小化公钥 + 签名的总规模。尽管设计者声称这一需求在某些情况下（如证书链）是合理的，但在大多数协议中，每条消息或者仅包含一个签名（无公钥），或者包含一个证书（一个公钥 + 一个签名）和一个额外的签名。因此，我们以稍大的公钥规模为代价，进一步减小签名规模。我们的签名规模远小于 ML-DSA，同时 $pk + 2 \cdot sig$ 的规模也小于 ML-DSA，这意味着在几乎所有情况下，我们的方案具有比 ML-DSA 更低的通信代价。

为达成此效率，与 ML-DSA 相比，我们移除了允许对公钥进行高压缩率的提示（hint）的使用。虽然这导致公钥规模较大，但我们避免了引入提示带来的两个缺点：第一，在签名中包含提示向量使签名规模增加约 5%；第二，提示向量使 MISIS 中的范数界加倍，从而导致更大的参数。

- **增强针对签名伪造的安全性。**通过观察到所有最优格求解算法均针对欧几里得范数求解，但我们的签名限制在无穷范数下定义，我们采用了一种简单但效果明显的方法来增强针对签名伪造的安全性：要求签名具有足够大的欧几里得范数，同时具备足够小的无穷范数，使得简单地在欧几里得范数下求解短格向量不再能导致签名伪造。这一新的困难假设称为 Shell-MISIS (Shell-CVP)，以体现所需的格向量位于高维立方体与高维球体之间的一个球壳中。

我们证明在这一新的困难假设下，与 ML-DSA 相比，我们的方案具有相似的安全比特数和更少的时间代价。但即使不考虑这一新假设的困难性，仅考虑 MISIS 下签名伪造的安全性，我们的方案仍能达到提案所要求的安全级别。

2. 算法描述

2.1 符号与记号

本节规定全文中使用的符号、运算符和记号约定。我们定义了基本数学集合（如 $\mathbb{Z}$ 、 $\mathbb{Z}_m$ 、 R_q ）、数据类型（如字节、字节串）和操作（如拼接、舍入、NTT 表示）。我们还建立了区分不同环的元素、向量、矩阵和数组的排版规则，以及索引、子数组和类型转换的约定。这些记号构成了后续算法描述和安全定义中使用的精确语言。

$a||b$ 两个比特/字节串 a 和 b 的拼接。

$a \odot b$ 环 T_q 中 a 与 b 的乘法。

$a \cdot b$ 或 ab 任何其他环中的乘法。

$a + b$ a 与 b 的加法。

a/b 整数除法。若不能假定 b 整除 a ，则使用 $\lfloor a/b \rfloor$ 或 $\lceil a/b \rceil$ 。

$\perp$ 表示算法失败或无输出的空白符号。

$\mathbb{B}$ 无符号 8 位整数（字节）的集合 $\{0, 1, \dots, 255\}$ 。

$\mathbb{Q}$ 有理数集。

$\mathbb{Z}$ 整数集。

$\mathbb{Z}_m$ 模 m 的整数环（即集合 $\{0, 1, \dots, m-1\}$ ，配备模 m 加法和乘法运算）。

$\mathbb{Z}_m^n$ $\mathbb{Z}_m$ 上的 n 元组集合，配备 $\mathbb{Z}_m$ -模结构。作为数据类型，它是长度为 n 的数组，其元素属于 $\mathbb{Z}_m$ 。

R 单变量多项式环 $\mathbb{Z}[X]/(X^n + 1)$ ，也记作 $\mathbb{Z}[X]/(X^n + 1)$ 。

R_q 单变量多项式环 $\mathbb{Z}_q[X]/(X^n + 1)$, 也记作 $\mathbb{Z}_q[X]/(X^n + 1)$, 由形如 $f = f_0 + f_1X + \cdots + f_{n-1}X^{n-1}$ 的多项式组成, 其中对所有 j 有 $f_j \in \mathbb{Z}_q$ 。

T_q R_q 在数论变换下的像。其元素称为 R_q 中多项式的“NTT 表示”。

S^* 若 S 是一个集合, 则表示 S 中元素的有限长度元组 (或数组) 的集合, 包括空元组。

S^k 若 S 是一个集合, 则表示 S 中元素的 k 元组 (或长度为 k 的数组) 的集合。

f_j 多项式 $f = f_0 + f_1X + \cdots + f_{n-1}X^{n-1} \in R_q$ 中 X^j 的系数。

$\hat{f}$ T_q 中等于多项式 $f \in R_q$ 的 NTT 表示的元素。

$\mathbf{v}^T, \mathbf{A}^T$ 行向量或列向量 $\mathbf{v}$ 的转置。一般地, 矩阵 $\mathbf{A}$ 的转置。

$B[i]$ 数组 B 中下标为 i 的元素。所有数组的下标从零开始。

$B[k:m]$ 数组 B 的子数组 $(B[k], B[k+1], \dots, B[m-1])$ 。

$|B|$ 若 B 是数字, 表示 B 的绝对值。若 B 是数组, 表示其长度。

$\lceil x \rceil$ x 的上取整 (大于或等于 x 的最小整数)。

$\lfloor x \rfloor$ x 的下取整 (小于或等于 x 的最大整数)。

$\lceil x \rceil$ x 四舍五入到最近整数。若 $x = y + 1/2$, 其中 $y \in \mathbb{Z}$, 则 $\lceil x \rceil = y + 1$ 。

$:=$ 表示左侧被定义为右侧。

$\leftarrow$ 表示左侧是从右侧均匀随机采样的元素。

环运算。 环 $\mathbb{Z}, \mathbb{Z}_q, \mathbb{Z}_2, R$, 和 R_q 的元素用斜体小写字母表示 (如 w)。环 T_q 的元素是 $\mathbb{Z}_q$ 中元素的长度为 n 的数组, 用带帽子符号的斜体字母表示 (如 $\hat{w}$)。 T_q 中元素的加法和乘法按逐元素执行。因此, T_q 的两个元素 $\hat{u}$ 和 $\hat{v}$ 的乘积的第 i 个元素为 $(\hat{u} \odot \hat{v})[i] = \hat{u}[i] \cdot \hat{v}[i] \in \mathbb{Z}_q$, 其中 T_q 中的乘法运算用符号 $\odot$ 表示。

当书写乘积 $a \cdot b$ 或和 $a + b$ 时, 若 a 或 b 是一个模 m 的同余类 (即若 a 或 b 是 $\mathbb{Z}_m$ 或 R_m 的元素), 则乘积或和也被理解为模 m 的同余类 (即 $\mathbb{Z}_m$ 或 R_m 的元素)。类似地, R 或 $\mathbb{Z}$ 的元素可以被“类型转换”为 R_m 或 $\mathbb{Z}_m$ 的元素, 并可作为作用于 R_m 或 $\mathbb{Z}_m$ 上函数的输入。在这两种情况下, 元素本身或其系数通过取包含该整数的唯一模 m 同余类从 $\mathbb{Z}$ 映射到 $\mathbb{Z}_m$ 。

R 或 R_m 的元素 w 的系数记作 w_i , 使得 $w = w_0 + w_1X + \cdots + w_{n-1}X^{n-1}$ 。若 w 在 R (相应地, R_m) 中且 t 在 $\mathbb{Z}$ (相应地, $\mathbb{Z}_m$) 中, 则 $w(t)$ 表示多项式在 $X = t$ 处的取值。

元素在 R 或 R_m 中的向量用粗体小写字母表示 (如 $\mathbf{v}$)。元素在 R 或 R_m 中的矩阵用粗体大写字母表示 (如 $\mathbf{A}$)。若 S 是一个环且 $\mathbf{v}$ 是 S 上的一个长度为 L 的向量, 则向量 $\mathbf{v}$ 中元素记作 $\mathbf{v}[0], \mathbf{v}[1], \dots, \mathbf{v}[L-1]$ 。 S 上 $K \times L$ 矩阵 $\mathbf{A}$ 的元素记作 $\mathbf{A}[i, j]$, 其中 $0 \leq i < K$ 且 $0 \leq j < L$ 。

S 上所有长度为 L 的向量的集合记作 S^L 。 S 上所有 $K \times L$ 矩阵的集合记作 $S^{K \times L}$ 。长度为 L 的向量也可视为 $L \times 1$ 矩阵。

2.2 签名方案概览

2.2.1 算法参数

本小节定义了算法中使用的主要记号与参数。算法的主要参数列举如下。

- q : 整数环 $\mathbb{Z}_q$ 的模数;
- n : 多项式模 $X^n + 1$ 的次数;
- k : MLWE 采样数;
- l : MLWE 秘密的维度;
- d : 公钥压缩使用的比特数;
- η_s, η_e : 秘密向量 $\mathbf{s}$ 和 $\mathbf{e}$ 的无穷范数界;
- γ_1 : 签名中随机性向量 $\mathbf{y}$ 系数的界;
- γ_2 : 拆分时定义低阶比特的参数 (与舍入相关);
- δ : 重缩放参数, 即 $\delta = \lfloor \gamma_2 / \gamma_1 \rfloor$;
- τ : 挑战向量 c' 中非零元素的个数;
- B : 与签名 $\mathbf{z}$ 相关的短向量的欧几里得范数界。

模约简。 对于偶 (相应地, 奇) 正整数 α , 我们定义 $r' = r \bmod^{\pm} \alpha$ 为唯一元素 r' , 使得 $r' \equiv r \pmod{\alpha}$, 且 r' 在范围 $-\frac{\alpha}{2} < r' \leq \frac{\alpha}{2}$ (相应地, $-\frac{\alpha-1}{2} \leq r' \leq \frac{\alpha-1}{2}$) 内。我们有时将其称为居中模 α 约简。对于任意正整数 α , 我们定义 $r' = r \bmod^+ \alpha$ 为唯一元素 r' , 使得 $r' \equiv r \pmod{\alpha}$ 且 r' 在范围 $0 \leq r' < \alpha$ 内。当精确表示不重要时, 我们简单写作 $r \bmod \alpha$ 。

元素的规模。 对于元素 $w \in \mathbb{Z}_q$, 我们用 $\|w\|_\infty$ 表示 $|w \bmod^\pm q|$ 。对于 $w = w_0 + w_1X + \dots + w_{n-1}X^{n-1} \in R$, 我们定义 ℓ_∞ 和 ℓ_2 范数:

$$\|w\|_\infty = \max_i \|w_i\|_\infty, \quad \|w\| = \sqrt{\|w_0\|_\infty^2 + \dots + \|w_{n-1}\|_\infty^2}.$$

类似地, 对于 $\mathbf{w} = (w_1, \dots, w_k) \in R^k$, 我们定义

$$\|\mathbf{w}\|_\infty = \max_i \|w_i\|_\infty, \quad \|\mathbf{w}\| = \sqrt{\|w_1\|^2 + \dots + \|w_k\|^2}.$$

我们用 S_η 表示所有满足 $\|w\|_\infty \leq \eta$ 的元素 $w \in R$ 。我们用 $\tilde{S}_\eta$ 表示集合 $\{w \bmod^\pm 2\eta : w \in R\}$ 。注意 S_η 和 $\tilde{S}_\eta$ 非常相似, 除了 $\tilde{S}_\eta$ 不包含任何具有 $-\eta$ 系数的多项式。

2.2.2 NTT 域表示

我们的模数 q 选择使得在模 q 下存在一个 $2n$ 次本原单位根 r 。这意味着分圆多项式 $X^n + 1$ 模 q 可分解为线性因子 $X - r^i$, 其中 $i = 1, 3, 5, \dots, 2n-1$ 。由中国剩余定理, 我们的分圆环 R_q 因此同构于环 $\mathbb{Z}_q[X]/(X - r^i) \cong \mathbb{Z}_q$ 的乘积。在此环的乘积中, 乘法是逐点的, 因此很容易计算。同构

$$a \mapsto (a(r), a(r^3), \dots, a(r^{2n-1})) : R_q \rightarrow \prod_i \mathbb{Z}_q[X]/(X - r^i)$$

可以借助快速傅里叶变换快速计算。由于 $X^n + 1 = X^n - r^n = (X^{n/2} - r^{n/2})(X^{n/2} + r^{n/2})$, 可以先计算映射

$$\mathbb{Z}_q[X]/(X^n + 1) \rightarrow \mathbb{Z}_q[X]/(X^{n/2} - r^{n/2}) \times \mathbb{Z}_q[X]/(X^{n/2} + r^{n/2})$$

然后分别对两个次数小于 $n/2$ 的约简多项式继续, 注意 $X^{n/2} + r^{n/2} = X^{n/2} - r^{3n/2}$ 。当基域为有限域时, 快速傅里叶变换也称为数论变换 (NTT)。自然的快速 NTT 实现不以 $a(r), a(r^3), \dots, a(r^{2n-1})$ 的顺序输出系数向量。因此我们定义多项式 $a \in R_q$ 的 NTT 域表示 $\hat{a} = \text{NTT}(a) \in \mathbb{Z}_q^n$ 具有如我们的参考 NTT 所输出顺序的系数。具体来说,

$$\hat{a} = \text{NTT}(a) = (a(r_0), a(-r_0), \dots, a(r_{n/2-1}), a(-r_{n/2-1}))$$

其中 $r_i = r^{\text{brv}(n/2+i)}$, $\text{brv}(k)$ 为 $\log n$ 比特数 k 的比特逆序。由此记号, 且由同构性质, 我们有 $ab = \text{NTT}^{-1}(\text{NTT}(a) \odot \text{NTT}(b))$ 。对于向量 $\mathbf{y}$ 和矩阵 $\mathbf{A}$, 表示 $\hat{\mathbf{y}} = \text{NTT}(\mathbf{y})$ 和 $\hat{\mathbf{A}} = \text{NTT}(\mathbf{A})$ 意味着构成 $\hat{\mathbf{y}}$ 和 $\hat{\mathbf{A}}$ 的每个多项式 $\hat{y}_i$ 和 $\hat{a}_{i,j}$ 都在 NTT 域表示中。

2.2.3 哈希

本算法使用若干不同的算法将字符串 $\{0, 1\}^*$ 哈希到各种形式的域上。下面给出这些函数的高层描述。

哈希到球。 令 B_τ 表示 R 中具有 τ 个系数为 -1 或 1 、其余为 0 的元素的集合。我们有 $|B_\tau| = 2^\tau \cdot \binom{n}{\tau}$ 。对于我们的签名方案，我们需要一个密码学哈希函数，分两步哈希到 B_τ 。第一步使用一个第二原像稳固的密码学哈希函数将 $\{0,1\}^*$ 映射到域 $\{0,1\}^n$ 。第二步由一个 XOF 组成，将第一步的输出映射为 B_τ 的一个元素。这个两步过程的原因是第一步的输出允许对 B_τ 中的元素进行非常简单紧凑的描述（在 XOF 与纯随机函数不可区分的假设下）。我们将用于在 B_τ 中创建随机元素的算法有时称为 Fisher-Yates 洗牌的“由内向外”版本，其高层描述见图1。第二步中的 XOF 用于使用第一步的输出作为种子，创建此算法所需的随机性（在第 03 和 04 步中）。

```

SampleInBall $_\tau(\rho)$ 
01 初始化  $\mathbf{c} = c_0 c_1 \dots c_{n-1} = 00 \dots 0$ 
02 for  $i := n - \tau$  to  $n - 1$ 
03    $j \leftarrow \{0, 1, \dots, i\}$ 
04    $s \leftarrow \{0, 1\}$ 
05    $c_i := c_j$ 
06    $c_j := (-1)^s$ 
07 return  $\mathbf{c}$ 

```

图 1: 使用输入种子 ρ (和一个 XOF) 创建一个具有 τ 个 ± 1 和 $n - \tau$ 个 0 的随机 n 元素数组，以生成第 03 和 04 步中需要的随机性。

展开矩阵 $\mathbf{A}$ 。 函数 `ExpandA` 将均匀种子 $\rho \in \{0,1\}^n$ 映射到 NTT 域表示中的矩阵 $\mathbf{A} \in R_q^{k \times l}$ 。矩阵 $\mathbf{A}$ 仅用于乘法。因此，为更快实现，展开函数 `ExpandA` 不输出 $\mathbf{A} \in R_q^{k \times l} = (\mathbb{Z}_q[X]/(X^n + 1))^{k \times l}$ ，而是输出 $\hat{\mathbf{A}} \in T_q^{k \times l} = (\mathbb{Z}_q^n)^{k \times l}$ ，它被解释为 $\mathbf{A}$ 的 NTT 域表示。由于 $\mathbf{A}$ 需要均匀采样且 NTT 是同构，`ExpandA` 也需要在此表示中均匀采样。为兼容 COMPASS-KEM，若实现产生的 NTT 向量顺序与我们的参考 NTT 不同，则需要以非连续顺序采样系数。

采样均匀随机向量。 函数 `Sample` 以任意集合 S 和种子 ρ 为输入，输出 S 中一个均匀随机元素，即 $\text{Sample}(S, \rho) := x \leftarrow S$ 。

2.2.4 拆分与压缩

拆分函数将 $\mathbb{Z}_q$ 中的整数拆分为高阶比特和低阶比特。给定整数 $x \in \mathbb{Z}_q$ 和正整数 γ ，函数 `Split`(x, γ) 输出一对 (x_1, x_0) 使得

$$x = x_1 \cdot 2\gamma + x_0, \quad x_0 \in [-\gamma, \gamma].$$

拆分的目的是：当给定任意元素 $r \in \mathbb{Z}_q$ 和另一个小元素 $z \in \mathbb{Z}_q$ 时，我们希望能够在不已知 z 的情况下恢复 $r + z$ 的高阶比特。可以证明以下结果成立：

引理 1. 对于任意整数 $s \in \mathbb{Z}_q$ 和任意实数 $\beta > 0$ ，设 $(x_1, x_0) = \text{Split}(x, \gamma)$ 和 $(x'_1, x'_0) = \text{Split}(x + s, \gamma)$ ，若 $|s| \leq \beta$ 且 $|x_0| < \gamma - \beta$ ，则 $x_1 = x'_1$ 。

证明. $s = x + s - x = (x_1 - x'_1) \cdot 2\gamma + (x_0 - x'_0)$ 。由于 $|x'_0| \leq \gamma$ 且 $|x_0| \leq \gamma - \beta - 1$ ，有 $|x_0 - x'_0| \leq 2\gamma - \beta - 1$ 。若 $x_1 \neq x'_1$ ，则 $|x_1 - x'_1| \geq 1$ 。因此 $|(x_1 - x'_1) \cdot 2\gamma + (x_0 - x'_0)| \geq 2\gamma - (2\gamma - \beta - 1) = \beta + 1$ ，而 $|s| \leq \beta$ ，矛盾。 $\square$

满足上述条件的拆分函数不唯一。在我们的算法中，我们设 $p = \lceil \frac{q}{2\gamma+1} \rceil$ ，令 $x_1 = \lfloor \frac{p}{q} \cdot x \rfloor \bmod^+ p$ ，且 $x_0 = x - x_1 \cdot 2\gamma \bmod^\pm q$ 。

对于多项式或多项式向量输入，Split 独立应用于每个系数。

压缩与解压缩。 为减小公钥的规模，我们引入以下压缩和解压缩算法。对于元素 $x \in \mathbb{Z}_q$ ，压缩函数 $\text{Comp}_d(x)$ 定义为：

$$\text{Comp}_d(x) = \begin{cases} \lfloor \frac{x-1}{2^d} \rfloor, & \text{若 } x > 0, \\ \lfloor \frac{x}{2^d} \rfloor, & \text{否则.} \end{cases}$$

对于多项式 $x \in R_q$ （或多项式向量）， Comp_d 按系数逐一应用。

解压缩函数 $\text{Decomp}_d(x)$ （对于 $x \in \mathbb{Z}_q$ ）返回

$$\text{Decomp}_d(x) = x \cdot 2^d + 2^{d-1},$$

且对多项式及多项式向量输入也按系数逐一应用。

2.3 核心算法

本节描述密钥生成、签名生成和签名验证的函数。在需要随机性的地方，随机值作为函数的输入提供。

2.3.1 密钥生成

密钥生成算法 COMPASS-SIG.KeyGen 以一个 n 比特（ $n/8$ 字节）的随机种子为输入，输出一个公钥和一个私钥，二者均编码为字节串。

种子 seed 按需使用记作 G 的密码哈希函数展开以产生其他随机值。具体来说：

- 一个 $n/8$ 字节的公开随机种子 ρ 。使用此种子，从 $R_q^{k \times l}$ 中伪随机采样多项式矩阵 $\mathbf{A} \in R_q^{k \times l}$ 。
- 一个 $n/8$ 字节的私有随机种子 σ 。使用此种子，从坐标多项式具有短系数的多项式向量集中伪随机采样多项式向量 $\mathbf{s} \in S_{\eta_s}^l$ 和 $\mathbf{e} \in S_{\eta_e}^k$ 。

核心密码学操作计算公开值

$$\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}.$$

向量 $\mathbf{t}$ 与矩阵 $\mathbf{A}$ 一起可被视为公钥的展开形式。向量 $\mathbf{t}$ 在实际公钥中被压缩，通过对每个系数丢弃 d 个最低有效比特，从而产生多项式向量 $\mathbf{t}_1$ 。此压缩是为性能优化，而非安全性。 $\mathbf{t}$ 的低阶比特可从少量签名中重建，因此不必视为秘密。

公钥 $\mathbf{pk}$ 是公开随机种子 ρ 和压缩多项式向量 $\mathbf{t}_1$ 的字节编码。私钥 $\mathbf{sk}$ 是主种子 seed 、秘密多项式向量 $\mathbf{s}$ 、噪声 $\mathbf{e}$ 以及 $\mathbf{t}_0$ 的字节编码，其中 $\mathbf{t}_0$ 包含由压缩引起的 $\mathbf{t}$ 的“丢失”比特。我们注意到，我们将主种子而不仅仅是公开种子包含在私钥中，这样私钥持有者可以验证 $\mathbf{s}, \mathbf{e}$ 是由算法 KeyGen 生成的正确的秘密密钥分量。这可以避免某些后门攻击。

描述：通过紧凑参数选择生成公/私钥对 $(\mathbf{pk}, \mathbf{sk})$ 。

输入： $\text{seed} \leftarrow \{0, 1\}^n$

1. $(\mathbf{pk}.\text{seed}, \mathbf{sk}.\text{seed}) := (\rho, \sigma) := G(\text{seed});$
 $\triangleright$ 通过哈希函数 G 派生 ρ ($\mathbf{pk}.\text{seed}$) 和 σ ($\mathbf{sk}.\text{seed}$)。
2. $\hat{\mathbf{A}} := \text{ExpandA}(\mathbf{pk}.\text{seed}) \in R_q^{k \times l};$
 $\triangleright$ 使用 ρ 在 NTT 域中构建矩阵
3. $(\mathbf{s}, \mathbf{e}) := \text{Sample}(S_{\eta_s}^l \times S_{\eta_e}^k, \mathbf{sk}.\text{seed}) \in R^l \times R^k;$
 $\triangleright$ 使用 σ 采样秘密 $\mathbf{s}$ 和噪声 $\mathbf{e}$ 。
4. $\mathbf{t} := \text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{s})) + \mathbf{e} \bmod q;$
5. $\triangleright$ 将私钥变换到 NTT 域，在 NTT 域中相乘并变换回时域。
6. $\mathbf{t}_1 := \text{Comp}_d(\mathbf{t}); \mathbf{t}_0 := \mathbf{t} - \text{Decomp}_d(\mathbf{t}_1); \triangleright$ 压缩公钥
7. $\mathbf{pk} := (\mathbf{pk}.\text{seed}, \mathbf{t}_1);$
8. $\mathbf{sk} := (\text{seed}, \mathbf{s}, \mathbf{e}, \mathbf{t}_0);$

输出：公/私钥对 $(\mathbf{pk}, \mathbf{sk})$

Algorithm 1: KeyGen

2.3.2 签名

算法 COMPASS-SIG.Sign 为一个消息 m 生成编码为字节串的数字签名。它以公钥 $\mathbf{pk} = (\mathbf{pk}.\text{seed}, \mathbf{t}_1)$ 、私钥 $\mathbf{sk} = (\text{seed}, \mathbf{s}, \mathbf{e}, \mathbf{t}_0)$ 和消息 m 为输入。算法输出签名 $\sigma = (\mathbf{z}, c)$ 。

签名算法过程如下。 COMPASS-SIG.Sign 首先使用确定性函数 ExpandA 展开公开种子 $\mathbf{pk}.\text{seed}$ 以获得矩阵 $\hat{\mathbf{A}} \in R_q^{k \times l}$ 。该矩阵是公开参数的一部分，从存储在公钥中的种子重建。然后，计算 $\mu = H(\mathbf{pk} \parallel m)$ ，其中 H 是密码学哈希函数。这将公钥和消息绑定在一起，通过后续步骤提供依赖消息的随机性。

此后， COMPASS-SIG.Sign 使用函数 Sample 从分布 $\tilde{S}_{\gamma_1}$ 采样生成随机向量 $\mathbf{y} \in R^l$ 。参数 γ_1 决定了 $\mathbf{y}$ 的无穷范数界。使用恢复的矩阵 $\mathbf{A} = \hat{\mathbf{A}}$ ，通过 NTT 操作 $\text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{y}))$ 计算乘积 $\mathbf{w} = \mathbf{A}\mathbf{y}$ 。我们注意到这是唯一使用 NTT 操作的地方，因为小多项式的乘法可以不用 NTT 高效完成。

函数 $\text{Split}(\mathbf{w}, \gamma_2)$ 被调用来基于界 γ_2 将 $\mathbf{w}$ 分解为高部分 $\mathbf{w}_1$ 和低部分 $\mathbf{w}_0$ 。这种拆分在基于格的签名中是典型的，用于管理误差项。

计算值 $c = H(\mu \parallel \mathbf{w}_1)$ ，即 μ 和高部分 $\mathbf{w}_1$ 拼接的哈希。然后 $c' = \text{SampleInBall}_\tau(c)$ 将此哈希转换为汉明重量为 τ （即恰好 τ 个非零系数，且为 ± 1 ）的向量 c' （或多项式）。

在计算初始候选值后，算法进入决定签名是否被接受的核心拒绝和一致性检查。首先，计算掩码后的秘密贡献 $\mathbf{z} = \mathbf{y} + c'\mathbf{s}$ ，并使用 $\text{Split}(\cdot, \gamma_2)$ 拆分 $\mathbf{w} - c'\mathbf{e}$ 为高部分 $\mathbf{u}_1$ 和低部分 $\mathbf{u}_0$ 。向量 $\mathbf{z}$ 本质上通过添加随机 $\mathbf{y}$ 隐藏秘密 $\mathbf{s}$ ，而 $\mathbf{w} - c'\mathbf{e}$ 的分解准备了验证所需的中间表示。

算法然后应用第一个拒绝条件：检查两个无穷范数界。若 $\|\mathbf{z}\|_\infty \geq \gamma_1 - \eta_s \cdot \tau$ 或 $\|\mathbf{u}_0\|_\infty \geq \gamma_2 - \eta_e \cdot \tau$ ，候选值被认为不安全或可能泄露秘密密钥信息，算法立即跳回第 3 行以采样一个新鲜的 $\mathbf{y}$ 并重启整个过程。若这些界得到满足，评估第二个拒绝条件以确保高比特一致性。算法计算另一个拆分： $(\mathbf{w}'_1, \mathbf{w}'_0) = \text{Split}(\mathbf{w} - c'\mathbf{e} + c'\mathbf{t}_0, \gamma_2)$ 。然后检查 $\mathbf{w}'_1$ 是否与 $\mathbf{u}_1$ 不同。任何不匹配表明重建值的高比特与早期拆分的高比特不一致；这将导致验证失败，因此算法再次回到第 3 行重启。

接下来，应用第三个拒绝条件以进一步避免签名伪造。算法测试是否 $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 < B \cdot \gamma_2^2$ 。若不等式成立，算法将此解释为不希望出现的（即范数过小）可能削弱安全性的情况，因此再次从第 3 行重启。

仅当所有三个拒绝检查均通过——即范数在允许范围内、高比特一致且欧氏范数条件未被触发——算法才最终确定签名。它输出 $\sigma = (\mathbf{z}, c)$ 作为消息 m 的最终签名。在实践中，整个过程可能重复多次直到找到有效签名，确保正确性和抵抗侧信道或泄露攻击。

总之，该算法遵循基于格的 Fiat-Shamir 签名中常见的“承诺-挑战-响应”模式。它反复采样随机性、构建候选签名并应用多个拒绝测试，以确保结果签名既有效又不泄露秘密密钥信息。利用拆分和对低部分的拒绝条件对于实现紧致安全证明和高效参数至关重要。

输入: $\text{pk} = (\text{pk.seed}, \mathbf{t}_1)$; $\text{sk} = (\text{seed}, \mathbf{s}, \mathbf{e}, \mathbf{t}_0)$; 消息 m ; 随机种子 ρ .

1. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\text{pk.seed}) \in R_q^{k \times l}$.
▷ 从 pk.seed 恢复 $\hat{\mathbf{A}}$.
 2. $\mu = H(\text{pk} \| m)$.
 3. $(\mathbf{y}, \rho) := \text{Sample}(\tilde{S}_{\gamma_1}^l \times \{0, 1\}^n, \rho)$; ▷ 随机种子 ρ 被更新.
 4. $\mathbf{w} := \text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{y})) \in R_q^k$.
 5. $(\mathbf{w}_1, \mathbf{w}_0) := \text{Split}(\mathbf{w}, \gamma_2)$;
 6. $c = H(\mu \| \mathbf{w}_1)$; $c' = \text{SampleInBall}_\tau(c)$.
 7. $\mathbf{z} := \mathbf{y} + c' \mathbf{s}$; $(\mathbf{u}_1, \mathbf{u}_0) := \text{Split}(\mathbf{w} - c' \mathbf{e}, \gamma_2)$;
 8. **If** $\|\mathbf{z}\|_\infty \geq \gamma_1 - \eta_s \cdot \tau$ **Or** $\|\mathbf{u}_0\|_\infty \geq \gamma_2 - \eta_e \cdot \tau$:
 9. **Goto Line #3.**
 10. **End If**
 11. $(\mathbf{w}'_1, \mathbf{w}'_0) := \text{Split}(\mathbf{w} - c' \mathbf{e} + c' \mathbf{t}_0, \gamma_2)$;
 12. **If** $\mathbf{w}'_1 \neq \mathbf{u}_1$:
 13. **Goto Line #3.**
 14. **End If**
 15. **If** $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 < B \cdot \gamma_2^2$:
 16. **Goto Line #3.**
 17. **End If**
- 输出签名** $\sigma = (\mathbf{z}, c)$.

Algorithm 2: $\text{Sign}(\text{pk}, \text{sk}, m)$

2.3.3 验证

验证算法 $\text{COMPASS-SIG.Verify}$ 以公钥 $\text{pk} = (\text{pk.seed}, \mathbf{t}_1)$ 、一个 n 比特的消息 m 和一个候选签名 σ 为输入。其目的是判定 σ 在给定公钥下是否为 m 的有效签名。

算法首先使用函数 ExpandA 确定性地展开种子 pk.seed 以重建矩阵 $\hat{\mathbf{A}} \in R_q^{k \times l}$ ，这与签名生成期间使用的矩阵完全相同。然后计算哈希值 $\mu = H(\text{pk} \| m)$ ，其中 H 是密码学哈希函数；这将公钥和消息绑定在一起以供后续验证。从签名 σ （假定为 $(\mathbf{z}, c)$ 的形式）中，算法提取挑战哈希 c ，并通过调用 $\text{SampleInBall}_\tau(c)$ 将其转换为汉明重量为 τ 的稀疏向量（或多项式） c' 。此步骤反映了签名算法内部执行的变换，确保使用相同的挑战表示。

核心验证步骤重建了应当与签名者原始承诺匹配的某个高部分向量。算法计算乘积 $\mathbf{A}\mathbf{z}$ （其中 $\mathbf{A}$ 是展开的矩阵）并从中减去 $c' \cdot \text{Decomp}_d(\mathbf{t}_1)$ 。这里 Decomp_d 是一个解压缩函数，将压缩的公钥分量 $\mathbf{t}_1$ 展开为全尺寸多项式，即 $\mathbf{t}_1 \cdot 2^d + 2^{d-1}$ ，因此 $c' \cdot \text{Decomp}_d(\mathbf{t}_1) = c' \cdot (\mathbf{t}_1 \cdot 2^d + 2^{d-1})$ 。结果向量随后使用 Split 函数以签名期间使用的相同界 γ_2 拆分为高部分 $\mathbf{w}'_1$ 和低部分 $\mathbf{w}'_0$ 。在正确生成的签名中， $\mathbf{w}'_1$ 应等于签名者用于派生挑战 c 的 $\mathbf{A}\mathbf{y}$ 的高部分。

在获得 $(\mathbf{w}'_1, \mathbf{w}'_0)$ 后, 算法同时评估三个条件。第一, 检查 $\mathbf{z}$ 的无穷范数严格小于 $\gamma_1 - \eta_s \cdot \tau$, 即 $\|\mathbf{z}\|_\infty < \gamma_1 - \eta_s \cdot \tau$ 。这是签名过程中第一个拒绝条件的反面, 保证 $\mathbf{z}$ 不会太大, 从而防止秘密信息的泄露。第二, 验证 $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 \geq B \cdot \gamma_2^2$, 这是第三个拒绝条件的反面; 此不等式确保签名满足正确性所需的质量界。第三, 重新计算 μ 与恢复的高部分 $\mathbf{w}'_1$ 拼接的哈希, 并检查结果是否等于签名中的挑战 c , 即 $H(\mu \parallel \mathbf{w}'_1) = c$ 。仅当所有三个条件同时成立—— $\mathbf{z}$ 上的范数界、 $(\mathbf{z}, \mathbf{w}'_0)$ 上的二次界以及哈希匹配——算法才输出“通过”。否则输出“失败”, 表示签名无效。

输入: $\text{pk} = (\text{pk.seed}, \mathbf{t}_1)$; $m \in \{0, 1\}^*$; 签名 σ 。

1. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\text{pk.seed}) \in R_q^{k \times l}$;
▷ 从 pk.seed 恢复 $\hat{\mathbf{A}}$ 。
2. $\mu = H(\text{pk} \parallel m)$;
3. $c' = \text{SampleInBall}_\tau(c)$;
 $(\mathbf{w}'_1, \mathbf{w}'_0) = \text{Split}(\text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{z})) - c' \cdot \text{Decomp}_d(\mathbf{t}_1))$;
4. **If** $\|\mathbf{z}\|_\infty < \gamma_1 - \eta_s \cdot \tau$ **And** $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 \geq B \cdot \gamma_2^2$ **And** $H(\mu \parallel \mathbf{w}'_1) = c$:
5. **Output Pass.**
6. **End If**

Output Fail.

Algorithm 3: $\text{Verify}(\text{pk}, m, \sigma)$

3. 设计原理

3.1 安全基础

本方案的理论安全性基于模格上的困难假设, 包括 MLWE、MSIS 以及一个新提出的困难假设 SelfTargetMSIS^R 。

定义 1 (MLWE). 对于整数 k, l, q, n 和实数 η_s, η_e , 我们称 $\mathcal{A}$ 在环 $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ 上求解 MLWE 问题的优势为

$$\text{Adv}_{k, l, \eta_s, \eta_e, q, n}^{\text{MLWE}}(\mathcal{A}) := \left| \Pr[\mathcal{A}(\mathbf{A}, \mathbf{t}) = 1 \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; \mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) = 1 \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; \mathbf{s} \xleftarrow{\$} [\eta_s]^{nl}; \mathbf{e} \leftarrow [\eta_e]^{nk}] \right|.$$

定义 2 (MSIS). 对于整数 k, l, q, n 和界 γ, γ' , 我们称 $\mathcal{A}$ 在环 $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ 上求解 MSIS 问题的优势为

$$\text{Adv}_{k, l, \gamma, \gamma', q, n}^{\text{MSIS}}(\mathcal{A}) := \Pr \left[\mathcal{A}(\mathbf{A}) = (\mathbf{z}, \mathbf{u}) \wedge \mathbf{A}\mathbf{z} = \mathbf{u} \bmod q \wedge \|\mathbf{z}\|_\infty \leq \gamma \wedge \|\mathbf{u}\|_\infty \leq \gamma' \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l} \right].$$

定义 3 (SelfTargetMSIS^R). 假设 $H : \{0, 1\}^* \mapsto B_\tau$ 是一个密码学哈希函数。对于整数 k, l, q, n, τ 和界 γ, γ' , 我们称 $\mathcal{A}$ 在环 $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ 上求解 SelfTargetMSIS^R 问

题的优势为

$$\text{Adv}_{k,l,\gamma,\gamma',q,n,\tau}^{\text{SelfTargetMSIS}^R}(\mathcal{A}) := \Pr \left[\begin{array}{l} \mathcal{A}(\mathbf{A}, \mathbf{t}) = (\mathbf{z}, \mathbf{u}, c, \mu) \wedge \|\mathbf{z}\|_\infty \leq \gamma \wedge \|\mathbf{u}\|_\infty \leq \gamma' \\ \wedge H(\mu \| (\mathbf{A}\mathbf{z} + \mathbf{u} - c' \cdot \text{Decomp}_d(\text{Comp}_d(\mathbf{t})))) = c \end{array} \middle| \begin{array}{l} \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; \\ \mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k \end{array} \right],$$

其中 $c' = \text{SampleInBall}_\tau(c)$ 。

在 [KLS18] 中, 一个名为 SelfTargetMSIS 的非标准困难假设被提出并用于 ML-DSA 的安全证明。作者证明了在经典随机预言机模型下存在从 MSIS 到 SelfTargetMSIS 的归约。与 SelfTargetMSIS 相比, 我们新定义的困难假设仅将 $\mathbf{t}$ 替换为 $\text{Decomp}_d(\text{Comp}_d(\mathbf{t}))$ 。但由于 $\mathbf{t}$ 的随机性不参与归约, 我们的新问题 SelfTargetMSIS^R 具备从 MSIS 的相同归约, 因此我们声明, 从可证明安全的角度来看, 这一新的困难假设的可靠性与 SelfTargetMSIS 相同。

3.2 安全模型

我们回顾签名方案的 (S)UF-CMA 安全性。

定义 4 (UF-CMA). 对于签名方案 $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$, $\mathcal{A}$ 的优势定义为

$$\text{Adv}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A}) := \Pr[\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A}) = 1],$$

其中 $\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A})$ 定义于图 2。若对于任何 PPT 敌手 $\mathcal{A}$ 有 $\text{Adv}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A}) \leq \text{negl}(\lambda)$, 则称 SIG 是 UF-CMA 安全的。

$\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A})$	$\mathcal{O}_{\text{Sign}}(\text{sk}, \mu)$
• $Q \leftarrow \emptyset$ • $(\text{pk}, \text{sk}) \xleftarrow{\\$} \text{KeyGen}(1^\lambda)$ • $(\mu^*, \text{sig}^*) \xleftarrow{\\$} \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\text{sk}, \cdot)}(\text{pk})$ • 返回 $\mu^* \notin Q \wedge \text{Verify}(\text{pk}, \mu^*, \text{sig}^*) = 1$	• $\text{sig} \xleftarrow{\\$} \text{Sign}(\text{sk}, \mu)$ • $Q \leftarrow Q \cup \{\mu\}$ • 返回 sig

图 2: 签名的 UF-CMA 安全性

定义 5 (SUF-CMA). 对于签名方案 $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$, $\mathcal{A}$ 的优势定义为

$$\text{Adv}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A}) := \Pr[\text{Exp}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A}) = 1],$$

其中 $\text{Exp}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A})$ 定义于图 3。若对于任何 PPT 敌手 $\mathcal{A}$ 有 $\text{Adv}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A}) \leq \text{negl}(\lambda)$, 则称 SIG 是 SUF-CMA 安全的。

$\text{Exp}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A})$	$\mathcal{O}_{\text{Sign}}(\text{sk}, \mu)$
• $Q \leftarrow \emptyset$ • $(\text{pk}, \text{sk}) \xleftarrow{\\$} \text{KeyGen}(1^\lambda)$ • $(\mu^*, \text{sig}^*) \xleftarrow{\\$} \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\text{sk}, \cdot)}(\text{pk})$ • 返回 $(\mu^*, \text{sig}^*) \notin Q \wedge \text{Verify}(\text{pk}, \mu^*, \text{sig}^*) = 1$	• $\text{sig} \xleftarrow{\\$} \text{Sign}(\text{sk}, \mu)$ • $Q \leftarrow Q \cup \{\mu, \text{sig}\}$ • 返回 sig

图 3: 签名的 SUF-CMA 安全性

3.3 参数集

COMPASS-SIG 是一种基于格密码的数字签名方案，其安全性依赖于模带误差学习问题 (MLWE)、模短整数解 (MSIS) 变种的困难性。为适应不同的安全需求，我们提供了四组参数集，分别针对 128、256、384 和 512 比特安全性。所有参数集均使用素数模数 q 、环次数 n 为 256 或 512，以及适当的矩阵维度 $(k \times l)$ 。对每个参数集，秘密向量 $\mathbf{s}$ 和误差向量 $\mathbf{e}$ 的系数取自 $\{-1, 0, 1\}$ (即 $\eta_s = \eta_e = 1$)，这提高了实现效率并有助于控制扩展因子。参数 d 定义了用于压缩和解压缩的比特数，减小了公钥规模。 γ_1 是随机向量 $\mathbf{y}$ 的无穷范数界，而 γ_2 作为拆分界，直接决定低比特的范围 (此处 γ_2 严格由 $(q-1)/\{4, 8, 16\}$ 给出，取决于安全级别)。重缩放因子 δ 在所有安全级别下固定为 4。挑战向量 $\mathbf{c}'$ 的汉明重量 τ 随安全级别的提高从 30 增加到 120，确保足够的难度抵御穷举攻击。参数 B 用于签名紧凑性检查，通过设置 $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2$ 的下界来平衡效率和安全性。所有参数经过精心选择，以保证适中的签名规模、正确的验证以及抵御已知格攻击 (如对 LWE 的原始攻击和对偶攻击，以及对 ISIS 的受限 CVP 求解器)。具体数值列于下表中。

表 1: COMPASS-SIG 参数集

安全性 (比特)	q	n	k	l	η_s	η_e	d	γ_1	γ_2	τ	B
128	2081281	256	3	4	1	1	4	2^{15}	130080	30	560
256	2081281	256	7	7	1	1	5	2^{17}	520320	60	1120
384	8380417	512	5	6	1	1	5	2^{18}	1047552	78	1760
512	8380417	512	7	7	1	1	5	2^{19}	2095104	120	2240

我们给出所有安全级别的密钥和签名规模，以及与 ML-DSA 在相似安全级别下的对比。可以看出，虽然我们的方案具有较大的公钥规模，但签名规模小得多。

此外，可以看出 ML-DSA 中 $\text{pk} + 2 \cdot \text{sig}$ 的规模对于 128 和 256 比特安全分别为 6152 和 11846 字节，而我们的方案分别为 5824 和 11776 字节。因此即便消息中包含证书，我们的方案相比 ML-DSA 仍具有更低的通信代价。

表 2: COMPASS-SIG 的密钥和签名规模 (字节)

算法名称	安全比特	公钥	私钥	签名
COMPASS-SIG-128	128	1664	960	2080
ML-DSA-44	128	1312	2560	2420
COMPASS-SIG-256	256	3616	2144	4080
ML-DSA-87	256	2592	4896	4627
COMPASS-SIG-384	384	5792	3136	7344
COMPASS-SIG-512	512	7648	4608	9024

3.4 失败分析与重复次数

接下来, 我们简要讨论重复次数, 即签名算法每次迭代中的拒绝概率。我们注意到高重复次数不会导致失败, 仅增加签名算法的时间代价。

对于第一个拒绝条件 (第 8 行), 我们遵循 ML-DSA 中的讨论, 拒绝概率约为 $1 - \exp(-nl\tau\eta_s/\gamma_1 - nk\tau\eta_e/\gamma_2)$, 约 60% ~ 65%。

对于第二个拒绝条件 (第 12 行), 我们仅需计算 $\mathbf{w} - c'\mathbf{e}$ 和 $\mathbf{w} - c'\mathbf{e} - c'\mathbf{t}_0$ 在高阶比特上不一致的概率, 这意味着 $\|\mathbf{u}_0 - c'\mathbf{t}_0\|_\infty \geq \gamma_2$, 其中 $\mathbf{u}_0$ 的系数均匀分布在 $[-\gamma_2 + \beta + 1, \gamma_2 - \beta - 1]$ 中, 而 $c'\mathbf{t}_0$ 中的每个系数是 τ 个独立值的和, 每个独立值均匀分布在 $[-2^{d-1}, 2^{d-1} - 1]$ 中。由于我们缺乏对此条件的解析表达式, 我们使用实验来确定合适的 d 值, 使得额外的拒绝率约为 5%。

第三个条件 (第 15 行) 的拒绝概率与 B 相关, 通过精心选择 B , 可使此步骤的拒绝概率接近于 0。

实验结果表明, 不同参数集的平均重复次数分别为 3.09、2.88、3.25、2.99。

接下来, 我们证明所有验证失败的签名都会在签名过程中被拒绝, 因此对于有效签名验证总是通过。

定理 1. 验证算法具有完美的正确性。

证明. 仅当三个条件检查中有一个失败时验证才会失败。

对于第一个条件, 即 $\|\mathbf{z}\|_\infty < \gamma_1 - \eta_s \cdot \tau$, 相同的条件在签名算法的第 8 行被检查。类似地, 第二个条件在签名算法的第 15 行被检查。

对于第三个条件, 我们有:

$$\mathbf{Az} - c' \cdot \text{Decomp}_d(\mathbf{t}_1) = \mathbf{A}(\mathbf{y} + c'\mathbf{s}) - c'(\mathbf{t} - \mathbf{t}_0) = \mathbf{Ay} + c'(\mathbf{As} - \mathbf{t}) + c'\mathbf{t}_0 = \mathbf{w} - c'\mathbf{e} + c'\mathbf{t}_0.$$

因此在签名算法和验证算法中的值 $\mathbf{w}'_1$ 相同, 这意味着 $\mathbf{w}'_1 = \mathbf{u}_1$ 。此外, 由于 $\|\mathbf{u}_0\|_\infty < \gamma_2 - \eta_e \cdot \tau$, 由引理 1, 我们有 $\mathbf{u}_1 = \mathbf{w}_1$ 。因此 $H(\mu\|\mathbf{w}'_1) = H(\mu\|\mathbf{w}_1) = c$ 总是成立。

可以看出, 签名算法仅在签名通过验证过程时才输出签名, 因此验证算法如果接收到有效签名则始终通过。□

3.5 差异性声明

我们的算法紧密遵循均匀分布下的标准 Fiat-Shamir with Abort 范式，这意味着该算法与使用相同范式的其他算法（典型的如 NIST 标准 ML-DSA）具有相似的结构。

然而，本算法的基本设计理念与 ML-DSA 有很大不同，这也导致了构造细节上的差异。本算法的最终目标是提供紧致的安全性估计并避免不必要的冗余。为实现此目标，我们不使用 ML-DSA 中的压缩-提示（compress-and-hint）技术。相反，我们在签名的欧几里得范数上添加了一个额外的拒绝条件，增强了针对签名伪造的安全性。

本算法的安全分析相比 ML-DSA 显示了更多差异。在理论分析中，我们引入了新的困难假设和界定最小熵的新方法。在具体分析中，我们基于新类型的格困难问题和新提出的格求解算法构建安全评估。这些使我们能够选择更小的参数以达到相同的安全级别，从而显著减小了签名规模。

4. 安全声明与分析

本方案的安全分析可以大部分继承自 ML-DSA。事实上，由于我们省略了 ML-DSA 中提示的使用，方案的安全分析得到了显著简化。本方案与 ML-DSA 的另一个不同之处是对欧几里得范数的额外限制，如签名算法第 16 行所示。然而，对签名的额外限制只会增强安全性，因为我们只需对其基本困难问题（即 SelfTargetMSIS^R ）添加额外的条件，这只会增强问题的困难性。下面，在理论安全分析中，我们简单地忽略此条件。然而，在具体安全分析中，我们将给出有关此额外限制对签名具体安全级别影响的更多细节。

4.1 理论安全性

定理 2. 设 $k, l, q, n, \tau, \eta_s, \eta_e, \gamma_1, \gamma_2$ 如算法中所定义， $\gamma := \gamma_1 - \eta_s \cdot \tau$ ， $\gamma' := \gamma_2 - \eta_e \cdot \tau$ 。

设 δ, ϵ 满足以下条件：在概率概率为 $1 - \delta$ 下， $\mathbf{w}_1$ 的最小熵至少为 $-\log \epsilon$ ，且 p 是签名算法每次迭代中的非中止概率。

则对于任何针对 COMPASS-SIG UF-CMA 安全性的敌手 $\mathcal{A}$ ，存在一个针对 MLWE 问题的敌手 $\mathcal{B}_1$ 、一个针对 SelfTargetMSIS^R 问题的敌手 $\mathcal{B}_2$ 和一个针对 MSIS 问题的敌手 $\mathcal{B}_3$ ，使得

$$\text{Adv}_{\text{COMPASS-SIG}}^{\text{UF-CMA}}(\mathcal{A}) \leq L + \text{Adv}_{k,l,\eta_s,\eta_e,q,n}^{\text{MLWE}}(\mathcal{B}_1) + \text{Adv}_{k,l,\gamma,\gamma',q,n,\tau}^{\text{SelfTargetMSIS}^R}(\mathcal{B}_2).$$

且

$$\text{Adv}_{\text{COMPASS-SIG}}^{\text{SUF-CMA}}(\mathcal{A}) \leq L + \text{Adv}_{k,l,\eta_s,\eta_e,q,n}^{\text{MLWE}}(\mathcal{B}_1) + \text{Adv}_{k,l,\gamma,\gamma',q,n,\tau}^{\text{SelfTargetMSIS}^R}(\mathcal{B}_2) + \text{Adv}_{k,l,2\gamma,2\gamma',q,n}^{\text{MSIS}}(\mathcal{B}_3).$$

在经典随机预言机模型下,

$$L = Q_S \epsilon \cdot \left(\frac{Q_S + 1}{2(1-p)^2} + \frac{Q_H}{1-p} \right) + \frac{Q_S Q_H \epsilon}{1-p} + \delta$$

; 在量子随机预言机模型下,

$$L = \frac{2Q_S \sqrt{\epsilon}}{1-p} \cdot \sqrt{Q_H + \frac{Q_S}{1-p}} + 2Q_H \sqrt{\frac{Q_S \epsilon}{1-p}} + \delta$$

, 其中 Q_S, Q_H 分别为对签名预言机和随机预言机的查询次数。

证明. 此证明紧密遵循已有工作 [BBD⁺23], 我们省略细节。 $\square$

接下来, 我们展示如何界定值 δ, ϵ 。由于 [KLS18] 和 [BBD⁺23] 中最小熵的原始计算远非紧致, 我们使用剩余哈希引理来界定 $\mathbf{w}_1$ 的最小熵。

由于在所有参数中我们有 $l \geq k$, 可以看出 $\{H_{\mathbf{A}}(\mathbf{y}) = \mathbf{w} := \mathbf{A}\mathbf{y} : \mathbf{A} \in R_q^{k \times l}\}$ 是一个通用哈希族。由此直接可得 $H'_{\mathbf{A}}(\mathbf{y}) = \mathbf{w}_1$ 也是通用的, 最小熵为 $-nk \log \frac{2\gamma_2+1}{q}$, 且 $\mathbf{y}$ 的最小熵为 $nl \log(2\gamma_1 - 2\eta_s \tau)$ 。由剩余哈希引理, 我们可以找到合适的值 κ (我们简单固定 $\kappa = 0.5$), 使得

$$\epsilon = (1 + \kappa) \left(\frac{2\gamma_2 + 1}{q} \right)^{nk},$$

且

$$\delta = \kappa^{-2} \cdot (2\gamma_1 - 2\eta_s \tau)^{-nl} \cdot \left(\frac{2\gamma_2 + 1}{q} \right)^{-nk}.$$

接下来, 我们给出在所有安全级别下最小化 L 值的 δ, ϵ 选择。为简便起见, 我们固定 $Q_S = 2^{80}$ 和 $Q_H = 2^\lambda$, λ 是安全参数。结果如下表所示, 其中 L_c 表示经典随机预言机模型下的归约损失, L_q 表示量子随机预言机模型下的归约损失。

表 3: 安全证明中的额外归约损失

安全级别	δ	ϵ	L_c	L_q
128	2^{-14000}	2^{-2300}	2^{-2090}	2^{-980}
256	2^{-30000}	2^{-1790}	2^{-1452}	2^{-597}
384	2^{-53000}	2^{-5118}	2^{-4652}	2^{-2133}
512	2^{-68000}	2^{-3582}	2^{-2988}	2^{-1237}

从表 3 可以看出, 在经典和量子模型下, 额外的归约损失均可忽略。

4.2 具体安全性

4.2.1 密钥恢复的具体安全性

任何 MLWE 实例都可以转换为规模为 $n \cdot k \times n \cdot \ell$ 的 LWE 实例，即秘密向量长度为 $n_{\text{LWE}} = n \cdot \ell$ 且 $m_{\text{LWE}} = n \cdot k$ 个样本。本节中我们重载记号，令 $(\mathbf{A}, \mathbf{t})$ 代表 LWE 样本而非 MLWE 样本。

原始攻击。原始攻击 [AGVW17] 使用 Kannan 嵌入技术 [Kan83] 将 LWE 问题归约为唯一最短向量问题 (uSVP)，然后使用格约简算法恢复目标向量。Albrecht 等人 [AFG13] 首先提出了针对 LWE 问题的原始攻击。

对于密钥恢复的安全性，攻击可以通过在给定 LWE 样本 $(\mathbf{A}, \mathbf{t})$ 的条件下寻找 LWE 秘密来执行。令 $\mathbf{t}' = \begin{pmatrix} \mathbf{t} \\ 0 \end{pmatrix}$ ，嵌入格的基构造如下：

$$\mathbf{B} = \begin{pmatrix} \mathbf{A}' & \mathbf{t}' \\ \mathbf{0}^T & 1 \end{pmatrix},$$

其中 $\mathbf{A}' = \begin{pmatrix} q\mathbf{I}_{m-n} & \mathbf{A} \\ \mathbf{0} & \mathbf{I}_n \end{pmatrix}$ 。在该嵌入格 $\mathbf{B}$ 中，存在唯一的格最短向量 $\mathbf{v} = (\mathbf{e}, \mathbf{s}, 1)$ ，

其中 $\mathbf{e}, \mathbf{s}$ 分别是 LWE 噪声向量和秘密向量。其范数约为 $\|\mathbf{v}\| \approx \sigma \sqrt{m_{\text{LWE}} + n_{\text{LWE}}}$ ，短于格第二短向量 $\lambda_2(\mathcal{L})$ 。因此，LWE 问题归约为嵌入格上的 uSVP 问题。用于评估嵌入格上 uSVP 的安全评估模型包括：Core-SVP 模型 [ADPS16]、格估算器中的 uSVP 模型以及两步模式 LWE 安全评估模型 [XWW+24]。此处我们依赖两步模式 LWE 安全评估模型 [XWW+24] 进行 LWE 原始攻击下的安全评估。研究表明，与独立 BKZ 模式 [SE94, CN11] 相比，两步模式可以更高效地求解 uSVP，从而优化 LWE 安全性的评估。

表 4: 两步模式下的具体安全参数（经典与量子参数设置）[XWW+24]

设置	安全级别	n_{LWE}	m_{LWE}	q	η_s	η_e	Memory($\log_2(\text{bit})$)	Time($\log_2(\text{gate})$)	d_{primal}	β_{BKZ}	β_{sieve}
经典	128	1024	768	2081281	1	1	105.05	151.61	1793	428	489
经典	256	1792	1792	2081281	1	1	193.75	276.01	3377	870	938
经典	384	3072	2560	8380417	1	1	324.95	461.78	5633	1529	1602
经典	512	3584	3584	8380417	1	1	386.44	549.35	6644	1840	1912
量子	80	1024	768	2081281	1	1	106.24	131.85	1793	425	494
量子	128	1792	1792	2081281	1	1	194.93	239.52	3377	867	944
量子	192	3072	2560	8380417	1	1	326.1	400.3	5633	1526	1607
量子	256	3584	3584	8380417	1	1	388	476.2	6644	1836	1920

为验证所提方案的安全性，我们在传统原始攻击路径（如 uSVP 和 BDD）下对 LWE 实例进行了具体参数评估。实验结果表明，在给定相同安全参数设置的情况下，传统攻击路径的计算开销（Time）高于表4中所示的**两步模式**。基于两步模式的评估框架更精

确地捕获了格约简过程中的安全瓶颈，确保每个安全级别的比特安全性稳健高于目标阈值。

我们还在相同安全级别下给出本方案与 NIST 标准 ML-DSA 的对比。

表 5: COMPASS-SIG 与 ML-DSA 在原始攻击下具体安全参数的对比。

安全级别	算法名称	Time($\log_2(\text{gate})$)	Memory($\log_2(\text{bit})$)
128-bit	COMPASS-SIG-128	151.61	99.1
128-bit	ML-DSA-44	150.6	104.4
256-bit	COMPASS-SIG-256	276.01	193.75
256-bit	ML-DSA-87	277.0	194.1

对偶攻击。最近工作 [CMHST25] 提出了 MATZOV 式对偶攻击的精华版本，并表明它可用于降低 Kyber (ML-KEM) 的安全估计。然而，他们的估计依赖许多不稳定的启发式假设，且未知是否适用于其他密码方案。为安全评估的完整性，我们也在 [CMHST25] 下给出本算法的安全评估，并表明评估结果也高于所需的安全级别。

表 6: 最新对偶攻击下的具体安全参数（经典与量子参数设置）[CMHST25]。

设置	安全级别	n_{LWE}	m_{LWE}	q	η_s	η_e	Time($\log_2(\text{gate})$)	d_{dual}	n_{enum}	n_{fft}	k_{fft}	β_{BKZ}	β_{sieve}
经典	128	1024	768	2081281	1	1	143.76	898	4	103	6	394	401
经典	256	1792	1792	2081281	1	1	256.53	1525	12	200	11	795	782
经典	384	3072	2560	8380417	1	1	423.91	2449	21	389	17	1389	1347
经典	512	3584	3584	8380417	1	1	519.04	2928	17	200	20	1727	1669
量子	80	1024	768	2081281	1	1	130.45	890	1	31	5	414	427
量子	128	1792	1792	2081281	1	1	225.45	1475	15	184	9	805	798
量子	192	3072	2560	8380417	1	1	371.88	2540	15	360	15	1404	1369
量子	256	3584	3584	8380417	1	1	450.45	2874	17	200	18	1727	1676

根据表6给出的评估结果，其中 $n_{\text{LWE}} = n \cdot l$ 、 $m_{\text{LWE}} = n \cdot k$ ，编码对偶攻击的时间开销估计显著低于相应的原始攻击估计。然而，这一比较应谨慎解读。编码对偶开销模型假定最终的子 LWE 求解步骤可忽略不计，而在实际密钥恢复场景中，此步骤可能需要求解维度为 $n - n_{\text{enu}}$ 的剩余 LWE 实例。若剩余实例通过原始攻击求解，其代价不应被忽略，尤其当 n_{enu} 较小时。因此，尽管报告的对偶攻击估计显著较低，该模型可能低估了完整秘密恢复的全部代价。

混合攻击。针对带误差学习 (LWE) 问题的混合攻击将格约简技术（如 BKZ）与中间相遇或猜测验证策略相结合，以利用小秘密或稀疏秘密向量的性质。这些攻击通常分为两类：

- **原始混合攻击 (bdd_hybrid)**: 如 [ACW19] 中所提, 该方法将秘密 $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2)$ 分区, 其中 $\mathbf{s}_2$ 被猜测, $\mathbf{s}_1$ 使用 BDD 求解器在降维格上恢复。通过利用高概率 BDD 求解器和动态维度选择, 平衡格预处理和译码的代价, 有效将高维问题转化为更易处理的低维问题。
- **对偶混合攻击**: 在 [EJK20] 中提出, 作为混合框架向对偶攻击的扩展。它拆分矩阵 $\mathbf{A}$ 和秘密 $\mathbf{s}$, 构造一个新的 LWE 样本, 维度降低但噪声 e' 增大。攻击然后通过递归枚举和分布区分器 (如 Levy 变换) 恢复部分秘密。

尽管这些攻击在特定参数集 (如稀疏秘密) 下具有理论优势, 但我们的全面安全评估表明, 对于本文考虑的参数设置, 这些混合攻击不会产生比标准原始攻击或对偶攻击更低的安全评估。

4.2.2 签名伪造的具体安全性

我们不直接考虑 SelfTargetMSIS^R 问题的具体困难性, 而是采用与 ML-DSA [SBN+20] 类似的方法: 首先选择一个承诺向量 $\mathbf{w}_1$, 计算 $H(\mu\|\mathbf{w}_1) = c$, 然后找到 $\mathbf{z}, \mathbf{u}$ 使得 $\mathbf{A}\mathbf{z} + \mathbf{u} = \mathbf{w}_1 \cdot (2\gamma_2) + c\mathbf{t}_1 \cdot 2^d$, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, $\|\mathbf{u}\|_\infty \leq \gamma_2$ 。注意由新困难问题 SelfTargetMSIS^R 的定义, $\mathbf{u}$ 的界简单地为 γ_2 , 无需引入如 ML-DSA 中 $2^{d-1} \cdot \tau$ 的附加项。令 $\mathbf{t}' = c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot (2\gamma_2)$, 问题可写成: $\mathbf{A}\mathbf{z} + \mathbf{u} = \mathbf{t}'$ 。

此问题与标准 MISIS 问题的唯一区别在于 $\mathbf{t}'$ 不是均匀随机向量。然而, 目前没有利用 $\mathbf{t}'$ 隐藏结构的方法, 此问题只能使用与求解标准 MISIS 问题相同的方法求解。对于此问题的困难性, 我们使用 [WVG25] 中给出的估算方法, 将无穷范数下的 MISIS 转化为称为受限 SVP 的 SVP 问题变体。我们注意到这是目前唯一在门计数模型下对 MISIS 的估算。

我们简要总结 [WVG25] 中的估算方法:

定义 6. 给定 $\mathcal{R}: \mathbb{R}^d \mapsto \{0, 1\}$ (称为限制), $\mathcal{L}$ 是一个 d 维格, 设 $\mathcal{P}(\text{len}) = \Pr(\mathcal{R}(\mathbf{v}) = 1 | \mathbf{v} \in \mathcal{L} \wedge \|\mathbf{v}\| \leq \text{len})$, 我们要求对任意 $\text{len} < \text{len}'$ 有 $\mathcal{P}(\text{len}) \geq \mathcal{P}(\text{len}')$ 。受限 SVP ($\text{RSVP}(\mathcal{L}, \mathcal{R})$) 旨在找到一个非零向量 $\mathbf{v} \in \mathcal{L}$ 使得 $\mathcal{R}(\mathbf{v}) = 1$ 。

我们称 $\mathcal{P}(\text{len})$ 为相关概率函数, 它描述了限制与格向量长度之间的关系。

类似地, 我们定义受限 CVP 问题: 对于一个目标向量 $\mathbf{t}$, 设 $\mathcal{P}_{\mathbf{t}}(\text{len}) = \Pr(\mathcal{R}(\mathbf{v}) = 1 | \mathbf{v} \in \mathcal{L} \wedge \|\mathbf{v} - \mathbf{t}\| \leq \text{len})$, 我们要求对任意 $\text{len} < \text{len}'$ 有 $\mathcal{P}(\text{len}) \geq \mathcal{P}(\text{len}')$ 。受限 CVP ($\text{RCVP}(\mathcal{L}, \mathcal{R})$) 旨在找到一个向量 $\mathbf{v} \in \mathcal{L}$ 使得 $\mathcal{R}(\mathbf{v}) = 1$ 。

为计算相关概率函数, 我们使用 [WVG25] 中以下引理, 该引理通过高维高斯来近似高维球体的均匀分布:

引理 2. (1) $\mathcal{B}^d(\text{len})$ 上的均匀分布接近于 d 维球面高斯 $\mathcal{N}\left(\mathbf{0}, \frac{\text{len}^2}{d} \cdot \mathbf{I}\right)$ 。

(2) 若 $\mathbf{v}$ 均匀分布在 $\mathcal{L} \cap \mathcal{B}^d(\text{len})$ 中, $\|\mathbf{v}\|_\infty \leq B$ 的概率约为 $\left(1 - 2\Phi\left(-\frac{\sqrt{d} \cdot B}{\text{len}}\right)\right)^d$, 其中 Φ 是标准高斯分布 $\mathcal{N}(0, 1)$ 的累积分布函数。

为求解 MISIS，我们首先使用重缩放方法构造如下格：

$$\mathbf{B}_\delta = \begin{pmatrix} q\mathbf{I} & \mathbf{A} \\ \mathbf{0} & -\delta\mathbf{I} \end{pmatrix},$$

其中 $\delta \approx \frac{\gamma_2}{\gamma_1 - \beta}$ ，且 $(\frac{\mathbf{u}}{\delta\mathbf{z}})$ 是其短向量之一。格中的相关概率可定义为： $\mathcal{P}(\text{len}) = \left(1 - 2\Phi\left(-\frac{\sqrt{d}\cdot\gamma_2}{\text{len}}\right)\right)^{n(k+l)}$ 。然后我们可以使用 Kannan 嵌入将受限 CVP 映射为受限 SVP，其细节见 [Kan83]。结果见表7。

批量近似 CVP 与近似 CVP。我们注意到，以伪造签名为目标，只需求解近似 CVP 的批量版本：给出许多不同的目标向量 $\mathbf{t}'_1, \dots, \mathbf{t}'_n$ ，找到其中任何一个的接近格向量。许多工作 [DLvW20, EK20, XWG25, KKNM25] 已表明批量 CVP 可能比 CVP 更容易求解。然而，我们声明相同的情况对近似 CVP 并不成立。原因很简单：对于批量 CVP 中使用的加速方法，首先使用快速多项式时间算法为每个 CVP 实例生成一个部分解，然后使用指数时间算法为这些部分解之一找到精确解。然而，近似 CVP 已经有指数多个可能的解而非一个，所以可以为单个近似 CVP 生成足够多的部分解，这意味着批量化无法导致加速。

增强近似 CVP $^\infty$ 的困难性。我们注意到，在签名算法中引入了一个额外的拒绝条件，即令向量 $\delta\mathbf{z}\|\mathbf{u}$ 具有足够大的欧几里得范数。由于 MISIS 可以通过近似 CVP $^\infty$ 求解，这个新问题可以通过如下定义的近似 CVP 变体求解：

定义 7 (Shell-CVP). 对于任意格 $\mathcal{L}$ 和目标向量 $\mathbf{t}$ ， $\text{Shell-CVP}_{B_\infty, B_2}$ 旨在找到一个格向量 $\mathbf{v} \in \mathcal{L}$ ，使得 $\|\mathbf{v} - \mathbf{t}\|_\infty \leq B_\infty$ 且 $\|\mathbf{v} - \mathbf{t}\|_2 > B_2$ 。

为便于后续讨论，我们定义此问题的绝对版本，即 B_∞ 和 B_2 是具体界而非与最短格向量长度相关的近似因子。我们还定义了其的对偶问题 Shell-SVP：

定义 8 (Shell-SVP). 对于任意格 $\mathcal{L}$ ， $\text{Shell-SVP}_{B_\infty, B_2}$ 旨在找到一个格向量 $\mathbf{v} \in \mathcal{L}$ ，使得 $\|\mathbf{v}\|_\infty \leq B_\infty$ 且 $\|\mathbf{v}\|_2 > B_2$ 。

由此直接得到，这个新问题比以 B_∞ 为界的近似 CVP $^\infty$ 更难，因为并非近似 CVP $^\infty$ 的所有解都是这个新问题的解。然而，估算这个新问题的具体困难性是一项困难的任务。我们首先考虑 Shell-SVP 而非 Shell-CVP 的困难性。具体来说，此问题不满足受限 SVP 的定义，因此受限 SVP 的估计不能适用。我们列出用于求解 Shell-SVP 的若干尝试：

(1) 使用多项式时间离散高斯采样算法，如 Klein 采样器或 Peikert 采样器，在离散高斯分布 D_σ 下采样大量格向量，在其中搜索目标向量，如 [Kan83] 所建议。我们可以省略采样的时间代价，复杂度因而以成功概率的逆为界，即 $(1 - 2\Phi(-\frac{B_\infty}{\sigma}))^{-d}$ 。然而，标准差 σ 必须足够大才能使多项式时间采样器适用。简单计算表明此方法效率不高。

(2) 将 BKZ 与无穷范数下的筛法相结合。此方法的主要问题是目前没有无穷范数格筛法的实现。唯一的理论分析表明，与欧几里得范数下 $2^{0.292d}$ 相比，无穷范数筛法复杂度约为 $2^{0.62d}$ 。因此此方法也效率不高。

(3) 选择一个合适的值 $B_2/\sqrt{d} < v < B_\infty$, 使得向量 $\mathbf{v} = \{v, \dots, v\}$ 位于球壳内, 然后通过求解以 $\mathbf{v}$ 为目标向量的受限 CVP (并通过 Kannan 嵌入进一步化为受限 SVP) 来找到球壳中的一个格向量。这被证实是最有效的方法, 且此方法的困难性估计可以由 [WXG25] 中的估算器经简单修改后给出。对于 Shell-CVP 而非 Shell-SVP, 我们只需将目标向量 $\mathbf{v}$ 改为 $\mathbf{v} + \mathbf{t}$, 这不改变时间复杂度。

我们需要解决的唯一问题是计算受限 CVP 问题的相关概率函数 $\mathcal{P}(\text{len})$ 。由引理2, 我们可以如下计算 $\mathcal{P}(\text{len})$:

对于 $\mathcal{P}(\text{len})$, 我们假设 $\mathcal{B}_2(\text{len}, \mathbf{v})$ 中的均匀随机向量 $\mathbf{x} = (x_1, \dots, x_d)$ 由 $\mathcal{N}\left(\mathbf{v}, \frac{\text{len}^2}{d} \cdot \mathbf{I}\right)$ 近似, 其中向量中的每个元素 x_i 服从 $\mathcal{N}\left(v, \frac{\text{len}^2}{d}\right)$ 。我们首先对 $\mathbf{x}$ 施加限制 $\|\mathbf{x}\|_\infty \leq B_\infty$ 。 $\mathbf{x}$ 通过此限制的概率为 $\left(\Phi\left(\frac{\sqrt{d} \cdot (B_\infty - v)}{\text{len}}\right) - \Phi\left(-\frac{\sqrt{d} \cdot (B_\infty + v)}{\text{len}}\right)\right)^d$ 。在我们的参数选择下, $\Phi\left(-\frac{\sqrt{d} \cdot (B_\infty + v)}{\text{len}}\right)$ 是可忽略的值, 因此我们将其简化为 $\Phi\left(\frac{\sqrt{d} \cdot (B_\infty - v)}{\text{len}}\right)^d$ 。

现在, 对于通过第一个限制的 $\mathbf{x}$, 我们可以看到每个 x_i 服从截断高斯分布, 即条件 $|x| \leq B_\infty$ 下的正态分布 $\mathcal{N}\left(\mathbf{v}, \frac{\text{len}^2}{d} \cdot \mathbf{I}\right)$ 。为表示 $\mathbf{x}$ 通过第二个限制 (即 $\|\mathbf{x}\| > B_2$) 的概率, 我们只需计算 $x_1^2 + \dots + x_d^2 > B_2^2$ 的概率。由于 d 足够大, 我们使用中心极限定理近似服从同一分布的 d 个独立变量之和, 即 $x_1^2 + \dots + x_d^2$ 近似服从 $\mathcal{N}(d\bar{\mu}, d\bar{\sigma}^2)$, 其中 $\bar{\mu}$ 和 $\bar{\sigma}^2$ 分别是截断分布的均值和方差。我们证明 $\bar{\mu}$ 和 $\bar{\sigma}^2$ 可以如下计算:

假设 x 在条件 $|x| \leq B$ 下服从分布 $\mathcal{N}(\mu, \sigma)$ 。为计算 $\bar{\mu} = \mathbb{E}(x^2 \mid |x| \leq B)$ 和 $\bar{\sigma}^2 = \mathbb{E}(x^4 \mid |x| \leq B) - (\mathbb{E}(x^2 \mid |x| \leq B))^2$, 我们考虑随机变量 $y = \frac{x - \mu}{\sigma}$ 在条件 $y \in [-\frac{B - \mu}{\sigma}, \frac{B - \mu}{\sigma}] := [a, b]$ 下服从分布 $\mathcal{N}(0, 1)$ 。则

$$\begin{aligned} \mathbb{E}(x^2 \mid |x| \leq B) &= \mathbb{E}((\sigma y + \mu)^2 \mid y \in [a, b]) \\ &= \sigma^2 M_2(a, b) + 2\sigma\mu M_1(a, b) + \mu^2, \\ \mathbb{E}(x^4 \mid |x| \leq B) &= \mathbb{E}((\sigma y + \mu)^4 \mid y \in [a, b]) \\ &= \sigma^4 M_4(a, b) + 4\sigma^3\mu M_3(a, b) + 6\sigma^2\mu^2 M_2(a, b) + 4\sigma\mu^3 M_1(a, b) + \mu^4, \end{aligned}$$

其中 $M_n(a, b)$ 是 $\mathcal{N}(0, 1)$ 限制在区间 $[a, b]$ 上的第 n 个原始矩 ($1 \leq n \leq 4$)。由分布 $\mathcal{N}(0, 1)$ 的密度函数 $\rho(x)$ 和累积函数 $\Phi(x)$, 我们得到

$$\begin{aligned} M_n(a, b) &= \mathbb{E}(x^n \mid x \leftarrow \mathcal{N}(0, 1), x \in [a, b]) \\ &= \frac{1}{\Phi(b) - \Phi(a)} \int_a^b x^n \rho(x) dx, \end{aligned}$$

其中 $\int_a^b x^n \rho(x) dx := c_n$ 可从递推公式 $c_n = (n - 1) \cdot c_{n-2} + a^{n-1} \rho(a) - b^{n-1} \rho(b)$ 推导得到, 初始条件为 $c_0 = \Phi(b) - \Phi(a)$ 和 $c_1 = \rho(a) - \rho(b)$ 。

因此我们给出了相关概率 $\mathcal{P}(\text{len})$ 的一个良好近似, 可用于签名伪造的安全估计。结果如下表所示, 括号中的数字是针对普通 MISIS $^\infty$ (即不考虑欧几里得范数界的近似 CVP $^\infty$) 的估计结果。

表 7: 签名中 SIS 问题的安全强度

安全级别	β_{BKZ}	经典时间 ($\log_2(\text{gate})$)	量子时间 ($\log_2(\text{gate})$)
128	409 (366)	145.6 (133.8)	136.62 (116.7)
256	913 (832)	287.2 (264.7)	248.55 (230.5)
384	1432 (1320)	433.5 (402.2)	376.3 (349.9)
512	1898 (1758)	565.4 (526.0)	495.3 (457.2)

从表7可以看出, 即便我们在安全估计中考虑近似 CVP 而非 Shell-CVP, 针对签名伪造的安全性仍能满足所需的安全级别, 而 Shell-CVP 中对欧几里得范数的额外限制将安全级别提高了 11.8 至 39.4 比特。

4.2.3 强签名伪造的具体安全性

我们注意到, 在安全证明中, 强签名伪造的附加条件被归约为找到一个短格向量 $\mathbf{z}' \parallel \mathbf{u}'$, 使得 $\|\mathbf{z}'\|_\infty \leq 2(\gamma_1 - \beta)$ 且 $\|\mathbf{u}'\|_\infty \leq 2\gamma_2$ 。然而, 不难理解找到这样的格向量并不始终导致签名伪造。

具体来说, 对于强签名伪造, 我们要求存在一个有效签名 $\mathbf{z}$, 且 $\mathbf{u}$ 是 $\text{Split}(\mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}')$ 中的低比特, 使得 $\mathbf{z}' - \mathbf{z}$ 和 $\mathbf{u}' - \mathbf{u}$ 构成有效签名, 即 $\|\mathbf{z}' - \mathbf{z}\|_\infty \leq \gamma_1 - \beta$ 且 $\|\mathbf{u}' - \mathbf{u}\|_\infty \leq \gamma_2$ 。然而, 对于一个仅满足 $\|\mathbf{z}'\|_\infty \leq 2(\gamma_1 - \beta)$ 和 $\|\mathbf{u}'\|_\infty \leq 2\gamma_2$ 的随机格向量, 即使我们执行 2^{80} 次签名查询 (提案中要求的上界), 这样的条件也很少成立。

因此, 要使强签名伪造有效, 我们必须对格向量 $\mathbf{z}' \parallel \mathbf{u}'$ 施加额外的限制, 这构成了受限 SVP 而非安全证明所隐含的近似 SVP。接下来我们计算限制函数, 并证明格向量通过此限制的概率过小, 以至于强签名伪造的复杂度不可能低于甚至接近普通签名伪造。由于我们在不可伪造性和强不可伪造性的攻击中使用基于相同格的受限 SVP 作为基础算法, 我们只需比较相关概率函数 $\mathcal{P}(\text{len})$ 。

为计算相关概率函数, 我们使用以下命题。

命题 1. 对于 $\mathbf{x} \xleftarrow{\$} \mathcal{B}_\infty^n(\mathbf{0}, a)$, 一个向量属于 $\mathcal{B}^n(\mathbf{x}, r)$ 且属于 $\mathcal{B}_\infty^n(\mathbf{0}, a)$ 的概率约为 $[1 - 2\Phi(-\frac{2a}{\sigma}) - \frac{\sigma}{a}(\rho(-\frac{2a}{\sigma}) - \rho(0))]^n$, 其中 $\sigma = \frac{r}{\sqrt{n}}$, $\rho(x)$ 和 $\Phi(x)$ 分别是分布 $\mathcal{N}(0, 1)$ 的密度函数和累积函数。

证明. 由引理2, 只需计算 $\mathbb{E}[\text{Pr}_\mathbf{x}]$, 其中

$$\begin{aligned} \text{Pr}_\mathbf{x} &= \Pr\left(\|\mathbf{x} + \mathbf{t}\|_\infty < a \mid \mathbf{t} \leftarrow \mathcal{N}(\mathbf{0}, \frac{r^2}{n}\mathbf{I})\right) \\ &= \prod_{i=1}^n \Pr\left(x_i + t_i < a \mid t_i \leftarrow \mathcal{N}(0, \frac{r^2}{n})\right). \end{aligned}$$

写 $\Pr_{x_i} = \Pr\left(x_i + t_i < a \mid t_i \leftarrow \mathcal{N}(0, \frac{r^2}{n})\right)$ 。由于 $\mathbf{x}$ 是均匀随机的, $\mathbb{E}[\Pr_{\mathbf{x}}] = \prod_{i=1}^n \mathbb{E}[\Pr_{x_i}] = \mathbb{E}[\Pr_x]^n$, 其中 $\Pr_x = 1 - \Phi\left(\frac{-a+x}{\sigma}\right) - \Phi\left(\frac{-a-x}{\sigma}\right)$ 。由性质 $\int \Phi(x)dx = x\Phi(x) + \rho(x)$, 有

$$\mathbb{E}[\Pr_x] = \frac{1}{2a} \int_{-a}^a \Pr_x dx = 1 - 2\Phi\left(-\frac{2a}{\sigma}\right) - \frac{\sigma}{a} \left(\rho\left(-\frac{2a}{\sigma}\right) - \rho(0)\right).$$

□

从以上表达式, 我们可以计算所有参数下的相关概率函数, 结果表明针对额外强签名伪造的相关概率远小于普通签名伪造的相关概率, 这意味着强伪造的额外要求对具体安全估计实际上几乎没有影响。

5. 性能

本节对所提 COMPASS-SIG 的性能特征和实际适用性进行全面评估。评估分为两个主要部分。首先, 在第5.1节中, 我们给出实现权衡的理论分析。研究了对称密码原语和拒绝采样机制的底层计算代价, 同时强调了系统级带宽优势——特别是由于我们改进的 MSIS 公式带来的私钥和签名规模的大幅减小。其次, 在第5.2节中, 报告从纯 ANSI C 参考实现获得的定量基准测试结果。通过给出 x86 架构上所有安全级别核心操作的详细周期计数分解, 我们证明所提方案不仅产生显著更紧凑的密钥规模, 而且在 Sign 和 Verify 过程中均实现了相比当前 ML-DSA 标准的大幅计算加速。

5.1 性能分析

5.1.1 实现考量与权衡

对称原语与拒绝采样。在 Fiat-Shamir with Aborts 签名中, 计算最密集的阶段是 Sign 过程中的迭代拒绝采样。每次迭代需要通过可扩展输出函数 (XOF) 展开掩码向量 $\mathbf{y}$, 执行 NTT 乘法, 并计算挑战哈希。因此, while 循环的平均迭代次数决定了整体签名延迟。通过精心调节欧几里得范数界 B 和 $\mathbf{z}$ 上的无穷范数限制, 我们的设计在不扩展参数维度的情况下确保了高效的接受率。

密钥/签名规模与带宽之间的权衡。本方案的一个核心设计理念是优化对实际网络性能最关键的参数, 即使以略微增加次要维度为代价。如经验数据所示, 我们的公钥 (PK) 略大于 ML-DSA (例如, 在 128 比特安全级别为 1664 字节对 1312 字节)。然而, 这种有意的权衡带来了私钥 (SK) 和签名规模的大幅减小。在 128 比特级别, 我们的私钥仅为 960 字节 (对比 ML-DSA-44 的 2560 字节), 签名规模压缩至 2080 字节 (对比 2420 字节)。

从工程和协议的角度来看, 这种非对称的规模特征提供了巨大的实用价值。在高频网络通信场景 (如传输层安全 TLS 握手) 中, 公钥通常是静态的, 可以通过 PKI 高效缓存或分发。相反, 签名本身是每次交互会话中必须计算和传输的动态有效载荷。因此,

最小化签名规模直接降低了每次事务的通信带宽，导致更低的传输延迟和减少的数据包分片。通过缩小签名实现的显著带宽节省远超存储稍大公钥的边际一次性代价。

5.1.2 跨安全级别的性能扩展

计算开销自然随四个安全级别的矩阵维度和多项式次数扩展。然而，签名稳定性的关键指标——拒绝采样迭代的期望次数——保持高度一致。

在 Fiat-Shamir with Aborts 范式中，算法性能对签名阶段的拒绝概率高度敏感。基于跨所有安全级别（从 128 比特到 512 比特）的 10,000 次签名生成的统计评估，`while` 循环的平均迭代次数保持显著稳定，范围在每次签名 2.88 至 3.25 次之间。这种一致性表明参数扩展完美地保留了算法效率，而不会在更高安全级别引发指数级拒绝惩罚。

此外，对拒绝条件的细粒度分析揭示了我们改进的模短整数解（MSIS）公式的效能。在我们的 `Sign` 过程中，欧几里得范数检查（签名算法第 15 行，检查界 B ）的拒绝率几乎为零——具体来说，在 128 比特级别为 0.0002，在更高级别为 0。这一经验证据表明我们改进的欧几里得范数界 B 经过了精心校准。它成功收紧了对高级格攻击的数学安全界，而没有无意中增加拒绝采样循环的计算负担，在理论安全鲁棒性和经验执行速度之间达到了最优平衡。

5.2 参考实现

在本节中，我们给出 COMPASS-SIG 的实现细节和性能评估。为提供全面评估，我们的评估涵盖两个不同的版本：纯 ANSI C 参考实现和利用 AVX2 指令的平台特定优化实现。参考实现旨在最大化可移植性和算法清晰度，提供一个无歧义的、与架构无关的基线以评估算法的固有计算效率。作为补充，AVX2 优化版本通过利用数据级并行，展示了现代 x86 平台上的高通量性能潜力。

5.2.1 x86 架构上的性能结果

为评估核心密码操作的串行执行效率，我们以 ML-DSA 为基准对 C 参考实现进行了基准测试。

基准测试方法 性能分析在一台配备 AMD Ryzen 7 8745HS 四核处理器和 8 GB RAM、运行 Ubuntu 22.04 的虚拟化工作站上进行。源代码使用 GCC 编译器以 `-O3` 优化标志编译以最大化编译器级效率。为确保周期计数测量的可靠性并减轻操作系统级别的噪声，禁用了 CPU 动态频率调整。报告的密钥生成（KeyGen）、签名（Sign）和验证（Verify）的时钟周期在 10,000 次独立执行上取平均值。

性能结果 表8总结了所提算法在四个安全级别上的平均时钟周期计数，并与 128 比特和 256 比特安全级别的相应 ML-DSA 参数进行了直接对比。

表 8: 签名算法性能对比 (x86 CPU 上的时钟周期)

算法	安全性 (比特)	KeyGen	Sign	Verify
ML-DSA-44 (C Ref)	128	331,703	1,474,335	375,036
COMPASS-SIG-128 (C Ref)	128	328,537	1,096,226	295,441
COMPASS-SIG-128 (AVX2)	128	156,890	384,936	151,876
ML-DSA-87 (C Ref)	256	931,215	3,380,411	975,866
COMPASS-SIG-256 (C Ref)	256	835,796	2,514,794	804,670
COMPASS-SIG-256 (AVX2)	256	378,248	894,835	367,472
COMPASS-SIG-384 (C Ref)	384	1,094,420	3,302,735	1,168,198
COMPASS-SIG-384 (AVX2)	384	541,841	1,428,775	591,102
COMPASS-SIG-512 (C Ref)	512	1,604,747	4,538,370	1,709,494
COMPASS-SIG-512 (AVX2)	512	728,644	1,695,434	709,480

周期分解与算法优势 基于表8中给出的经验数据, 我们提出的签名方案在所有核心操作阶段均展示了相对 ML-DSA 的深刻性能优势。

加速签名: 基于格签名的计算最昂贵操作是 Sign 过程, 很大程度上由于拒绝采样的迭代性质。与 ML-DSA 相比, 我们的算法在 128 比特和 256 比特安全级别均实现了约 $1.34\times$ 的显著加速。这一显著加速根本驱动于我们改进的模短整数解 (MSIS) 公式所衍生的优化数学界。通过大幅降低秘密密钥的维度和掩码向量的规模, 每次迭代的计算负担——特别是生成候选向量所需的 NTT 转换和对称 XOF 展开——被大幅最小化。此外, 由于我们精细调节的欧几里得范数界保持了稳定且较低的拒绝率, 算法成功享受了更小多项式运算的收益, 而不会遭受 while 循环重启次数增加的问题。

精简验证: Verify 函数在 128 比特和 256 比特分别展示了 $1.27\times$ 和 $1.21\times$ 的加速。由于 Fiat-Shamir 范式中的签名验证是无需拒绝且确定性的, 这一加速直接由算术复杂度的降低带来。我们的方案在显著更小的签名规模 (例如, 在 128 比特级别为 2080 字节签名对比 ML-DSA-44 的 2420 字节签名) 上运行。因此, 解析签名、执行逆向 NTT 操作以及重建承诺哈希所需的 CPU 周期更少, 且对缓存子系统的压力更低。

最终, 基线基准测试确认所提方案成功利用更紧致的 MSIS 假设, 不仅增强了理论安全性, 还实现了经验计算延迟的严格降低, 获得了相对现有标准的决定性性能领先。

5.2.2 AVX2 优化实现

为补充我们的可移植 ANSI C 参考实现, 我们开发了一个针对配备高级向量扩展 2 (AVX2) 指令集的现代 x86 处理器的高度优化实现, 如表8所示。我们没有依赖通用优化框架, 而是设计了一个针对本方案独特代数结构量身定制的 AVX2 架构。该方法充分利用数据级并行, 自主加速对称密码原语和核心多项式算术。

对称原语 在 Sign 和 KeyGen 过程中, 计算延迟的显著部分被用于矩阵展开和拒绝采样的可扩展输出函数 (XOF) 所消耗。为缓解此问题, 我们实现了高度并行的 4 路 Keccak-f[1600] 置换。通过利用 256 位 `_mm256i` 寄存器, 我们的实现同时评估四个独立的 Keccak 状态。这种并行架构大幅降低了生成展开矩阵 $\hat{\mathbf{A}}$ 和掩码向量 $\mathbf{y}$ 的延迟。

双参数多项式算术与 NTT AVX2 实现的一个显著工程亮点是其对双模数架构的原生支持。为覆盖全部四个安全级别, 我们的方案操作两组不同的参数集: 级别 1 和级别 2 使用 $q = 2081281, n = 256$; 级别 3 和级别 4 使用 $q = 8380417, n = 512$ 。

由于两个模数都需要至多 23 比特的精度, 多项式系数必须表示为 32 位有符号整数 (`int32_t`)。因此, 单个 256 位 AVX2 寄存器恰好打包 8 个系数, 实现 $8\times$ 并行因子用于逐元素向量操作。为在不引入分支惩罚的情况下最大化指令吞吐量, 我们独立设计了两套独立的高度展开的 AVX2 内联函数代码, 分别针对每个模数:

- **针对 $q = 2081281, n = 256$:** 我们构造了一套专用的向量化程序, 用于 8 层数论变换 (NTT)。这包括专门为 $q = 2081281$ 校准的定制 Montgomery 约简乘数和 Barrett 常数。Cooley-Tukey 蝶形运算利用 `_mm256_mul_epi32` 和 `_mm256_srli_epi64` 指令, 在向量寄存器内原生执行 32 位模运算。
- **针对 $q = 8380417, n = 512$:** 对于该高安全参数集, 我们设计了高级的 32 位 SIMD 约简策略, 并大幅扩展了内存访问模式和洗牌指令 (如 `_mm256_permutevar8x32_epi32`), 以在 9 层 NTT 中无缝处理更大的 $n = 512$ 多项式数组。这种独立的内存路由设计确保在扩展的内存占用上 L1 缓存带宽得到最优利用。

拒绝采样与数据打包 AVX2 指令集在加速拒绝采样循环——Fiat-Shamir 签名中最突出的瓶颈——中发挥了关键作用。在提取均匀有界系数或评估范数条件时, 我们使用 32 位向量化比较指令 (`_mm256_cmpgt_epi32`) 和混合操作 (`_mm256_blendv_epi8`)。这种自主机制允许算法以严格无分支的方式验证欧几里得范数 B 和无穷范数限制, 有效防止分支预测失败导致的流水线停顿。此外, 签名序列化的比特打包和解包例程从头开始完全向量化, 用高速 SIMD 移位和掩码替代了顺序按位操作。

6. 特性声明

在本节中, 我们对算法的特性做出声明。

- **创新性。** 在本提案中, 我们在理论和具体安全分析中提出了新颖的技术, 以展示 ML-DSA 参数选择中的冗余性。具体来说, 我们将具体安全分析中使用的重缩放方法与对最小熵的更精确计算相结合, 在不损害其不可伪造性的前提下最小化签名的规模。我们还给出了具体分析, 表明与 ML-DSA 设计者的指示不同, 强不可伪造性的额外要求不需要更大的参数。移除冗余使我们能够为基于 Fiat-Shamir with

Abort 的格签名提供一种新的设计，同时优化了签名规模以及所有算法的时间代价。除上述贡献点外，我们还提出了一个新的基于格的困难问题，即 Shell-CVP，进一步放大了针对签名伪造的安全性。

- 简洁性。本方案在设计上和实现上均满足简洁性。本方案遵循基于格的 Fiat-Shamir with Abort 标准范式，其安全归约已被充分研究。具体来说，与 ML-DSA 相比，我们移除了提示的使用，进一步简化了设计。本方案基于素数模数的模格上的困难问题，享有最坏情况安全性，且其中大多数具有已被充分研究的困难性。为缓解对新的困难问题 Shell-CVP 的担忧，我们证明即便不考虑 Shell-CVP 变体，近似 CVP 的困难性也足以达到安全级别。在实现上，我们避免使用高斯采样等重型操作，并证明本方案可以仅从包括 NTT、模乘和哈希函数的基本操作构造。
- 灵活性。除底层密码哈希函数外，本方案的实现仅依赖于模整数操作，可在软件和硬件平台上高效实现，并支持高级别的并行性，允许在效率和资源之间进行灵活平衡。
- 兼容性。我们还在 COMPASS 套件中提出了密钥封装方案，两个方案均基于模格，具有相同的多项式次数（虽然模数不同）。将 COMPASS-KEM 与 COMPASS-SIG 结合使用，可实现包括认证密钥交换在内的其他功能。此外，本方案具有比 ML-DSA 更短的签名规模，因此将本方案适配到现有协议可能比 ML-DSA 更容易。
- 可扩展性。已知基于格的签名，尤其是基于 Fiat-Shamir with Abort 的签名具有高度功能性，通过改变底层的零知识证明，Fiat-Shamir with Abort 签名方案可以处理复杂功能，包括群签名、环签名等。本方案可作为这些更高级方案的构造基础。
- 跨平台实现性能优势。本方案的基本操作单元是 $R_q/(X^n+1)$ 中的多项式 ($n = 256$ 或 $n = 512$)，与基于 RLWE 或 NTRU 的方案相比规模相对较小。虽然 NTT 和模乘操作均可轻松并行实现，但在 256 维或 512 维向量上的 NTT 操作对于资源受限情况也是可接受的。

参考文献

- [ACW19] Martin R Albrecht, Benjamin R Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In *International Conference on Selected Areas in Cryptography*, pages 467–491. Springer, 2019.
- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange—a new hope. In *25th USENIX security symposium (USENIX Security 16)*, pages 327–343, 2016.

- [AFG13] Martin R. Albrecht, Robert Fitzpatrick, and Florian Goepfert. On the efficacy of solving LWE by reduction to unique-SVP. Cryptology ePrint Archive, Paper 2013/602, 2013.
- [AGVW17] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
- [BBD⁺23] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 358–389. Springer, 2023.
- [CMHST25] Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 444–476, Cham, 2025. Springer Nature Switzerland.
- [CN11] Yuanmi Chen and Phong Q Nguyen. Bkz 2.0: Better lattice security estimates. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 1–20. Springer, 2011.
- [DLvW20] Léo Ducas, Thijs Laarhoven, and Wessel PJ van Woerden. The randomized slicer for cvpp: sharper, faster, smaller, batchier. In *IACR International Conference on Public-Key Cryptography*, pages 3–36. Springer, 2020.
- [EJK20] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a dual/hybrid approach to small secret lwe: A dual/enumeration technique for learning with errors and application to security estimates of the schemes. In *International Conference on Cryptology in India*, pages 440–462. Springer, 2020.
- [EK20] Thomas Espitau and Paul Kirchner. The nearest-colattice algorithm: Time-approximation tradeoff for approx-cvp. *Open Book Series*, 4(1):251–266, 2020.

- [Kan83] Ravi Kannan. Improved algorithms for integer programming and related lattice problems. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, STOC '83, page 193–206, New York, NY, USA, 1983. Association for Computing Machinery.
- [KKNM25] Alexander Karenin, Elena Kirshanova, Julian Nowakowski, and Alexander May. Fast slicer for batch-cvp: Making lattice hybrid attacks practical. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 100–132. Springer, 2025.
- [KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III*, Lecture Notes in Computer Science, pages 552–586. Springer, 2018.
- [SBN⁺20] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marcos Manzano, and Ramesh Karri. Crystals-dilithium. In *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 13–30. Springer, 2020.
- [SE94] Claus-Peter Schnorr and Martin Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical programming*, 66(1):181–199, 1994.
- [WXG25] Geng Wang, Wenwen Xia, and Dawu Gu. Heuristic algorithm for solving restricted svp and its applications. In *International Conference on Post-Quantum Cryptography*, pages 119–152. Springer, 2025.
- [XWG25] Wenwen Xia, Geng Wang, and Dawu Gu. Solve approximate cvp via variants of nearest-colattice. *Cryptology ePrint Archive*, 2025.
- [XWW⁺24] Wenwen Xia, Leizhang Wang, Geng Wang, Dawu Gu, and Baocang Wang. A refined hardness estimation of lwe in two-step mode. In *IACR International Conference on Public-Key Cryptography*, pages 3–35. Springer, 2024.